

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
Packages



Introduction

In Simple Words we can say that A package means “putting classes together”.

In a package we can define classes and interfaces.

Concept of package is also known as “Class Libraries” in other languages.

In other words we can say package is a “container for classes and interfaces”

Introduction

The classes contained in the packages of other programs can be easily reused.

Two classes in two different packages can have the same name. At that time we have to refer them with fully qualified name.

package also provides a way to hide classes also.

Types of Packages

Packages can be classified in two different types :

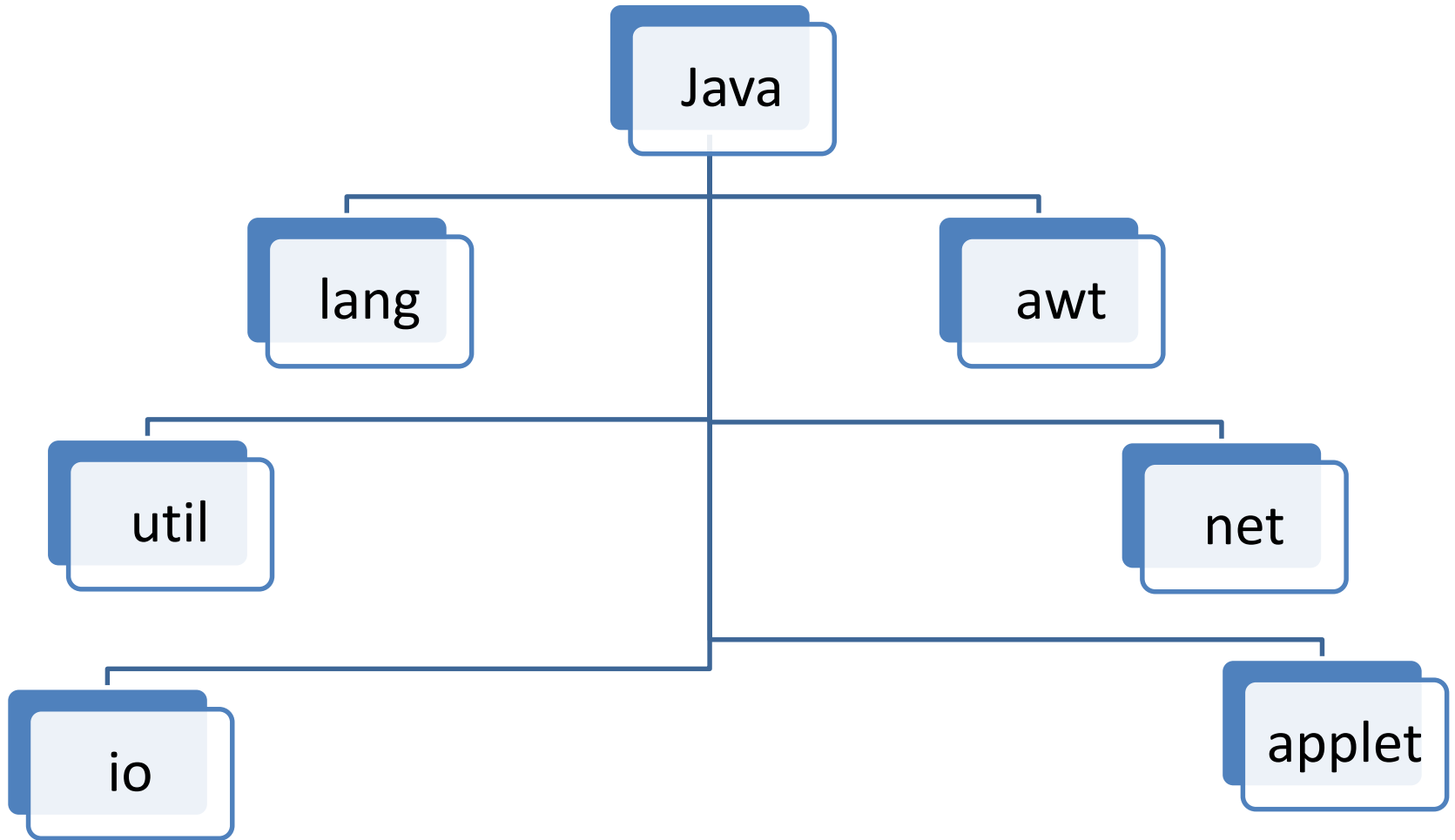
1.Java API Packages

2.User Defined Packages

Java API packages means readymade packages given by java.

User Defined packages means the packages developed by us.

Java API Packages



Java API Packages

Package Name	Description
java.lang	Language Support Classes. This package imported itself and includes classes for String, Maths, Threads and Exceptions
java.util	Language utility classes such as vectors, hash tables random numbers, date etc.
java.awt	Set of classes for implementing GUI
java.io	Input output classes – Provides facility for data input and output
java.net	Classes for networking.
java.applet	Classes for creating and implementing applets.

How to use API Packages

We can use Java built in packages by using import statement in our program. At the time of using import statement we have two options either we can import a particular class of the Package

import java.io.DataInputStream;

Or we can import All the classes of Package

import java.io.*;

Static import

By adding the word `static` after the `import` keyword, you change your import to a static one. when an import is static, all the accessible methods and variables of the class can be used without prefixing its access with the class name. For instance, if you static import the `java.awt.Color` class with `import static java.awt.Color.*;`, you just have to include `RED` in your code, instead of `Color.RED`. A simple version of this feature is as follows :

Static import

```
import static java.awt.Color.*;  
public class ImportTest  
{  
    public static void main(String args[])  
    {  
        System.out.println(RED);  
    }  
}
```

Java Naming Conventions

A naming convention is a rule to follow as you decide what to name your identifiers (e.g. class, package, variable, method, etc..).

Whenever we declare any new package, class, method or variable at that time we have to keep in mind these conventions.

Remember that these are not rules but these are only conventions for easy identification of the above mentioned things.

Need for Java Naming Conventions

Different Java programmers can have different styles and approaches to the way they program. By using standard Java naming conventions they make their code easier to read for themselves and for other programmers. Readability of Java code is important because it means less time is spent trying to figure out what the code does, leaving more time to fix or modify it.

Need for Java Naming Conventions

To illustrate the point it's worth mentioning that most software companies will have a document that outlines the naming conventions they want their programmers to follow. A new programmer who becomes familiar with those rules will be able to understand code written by a programmer who might have left the company many years before hand.

Need for Java Naming Conventions

When choosing a name for an identifier make sure it's meaningful. For instance, if your program deals with customer accounts then choose names that make sense to dealing with customers and their accounts (e.g., `customerName`, `accountDetails`). Don't worry about the length of the name. A longer name that sums up the identifier perfectly is preferable to a shorter name that might be quick to type but ambiguous.

Different Case Conventions

(1) Lowercase : is where all the letters in a word are written without any capitalization (e.g., while, if, mypackage).

(2) Uppercase : is where all the letters in a word are written in capitals. When there are more than two words in the name use underscores to separate them (e.g., MAX_HOURS, FIRST_DAY_OF_WEEK).

(3) CamelCase : (also known as Upper CamelCase) is where each new word begins with a capital letter (e.g., CamelCase, CustomerAccount, PlayingCard).

Different Case Conventions

(4) Mixed case : (also known as Lower CamelCase) is the same as CamelCase except the first letter of the name is in lowercase (e.g., hasChildren, customerFirstName, customerLastName).

Standard Java Naming Conventions

(1) Packages: Names should be in lowercase. With small projects that only have a few packages it's okay to just give them simple (but meaningful!) names: `package pokeranalyzer` `package mycalculator`. In software companies and large projects where the packages might be imported into other classes, the names will normally be subdivided. Typically this will start with the company domain before being split into layers or features: `package com.mycompany.utilities` `package org.bobscompany.application.userinterface`.

Standard Java Naming Conventions

(2) Classes: Names should be in CamelCase. Try to use nouns because a class is normally representing something in the real world: e.g.

class Customer

class Account

Standard Java Naming Conventions

(3) Interfaces: Names should be in CamelCase. They tend to have a name that describes an operation that a class can do: e.g.

interface Comparable

interface Enumerable

Note that some programmers like to distinguish interfaces by beginning the name with an "I": e.g.

interface IComparable

interface IEnumerable

Standard Java Naming Conventions

(4) Methods : Names should be in mixed case. Use verbs to describe what the method does: e.g.

```
void calculateTax()
```

```
string getSurname()
```

Standard Java Naming Conventions

(5) Variables: Names should be in mixed case. The names should represent what the value of the variable represents: e.g.

string firstName

int orderNumber

Only use very short names when the variables are short lived, such as in for loops: for (int i=0; i<20;i++) { //i only lives in here }

(6) Constants: Names should be in uppercase. e.g.

static final int DEFAULT_WIDTH

static final int MAX_HEIGHT

Creation of New Package

Syntax :

```
package <name of package>;  
public class <class name>  
{  
    .....  
        Body of the Class  
    .....  
}
```

Creation of New Package

Example :

```
package firstPackage;  
public class FirstClass  
{
```

.....

Body of the Class

.....

```
}
```

Creation of New Package

Suppose we have a file called HelloWorld.java, and we want to put this file in a package **world**. First thing we have to do is to specify the keyword **package** with the name of the package we want to use (**world** in our case) on top of our source file, before the code that defines the real classes in the package, as shown in our HelloWorld class below:

Creation of New Package

// only comment can be here

package world;

public class HelloWorld

{

public static void main(String[] args)

{

System.out.println("Hello World");

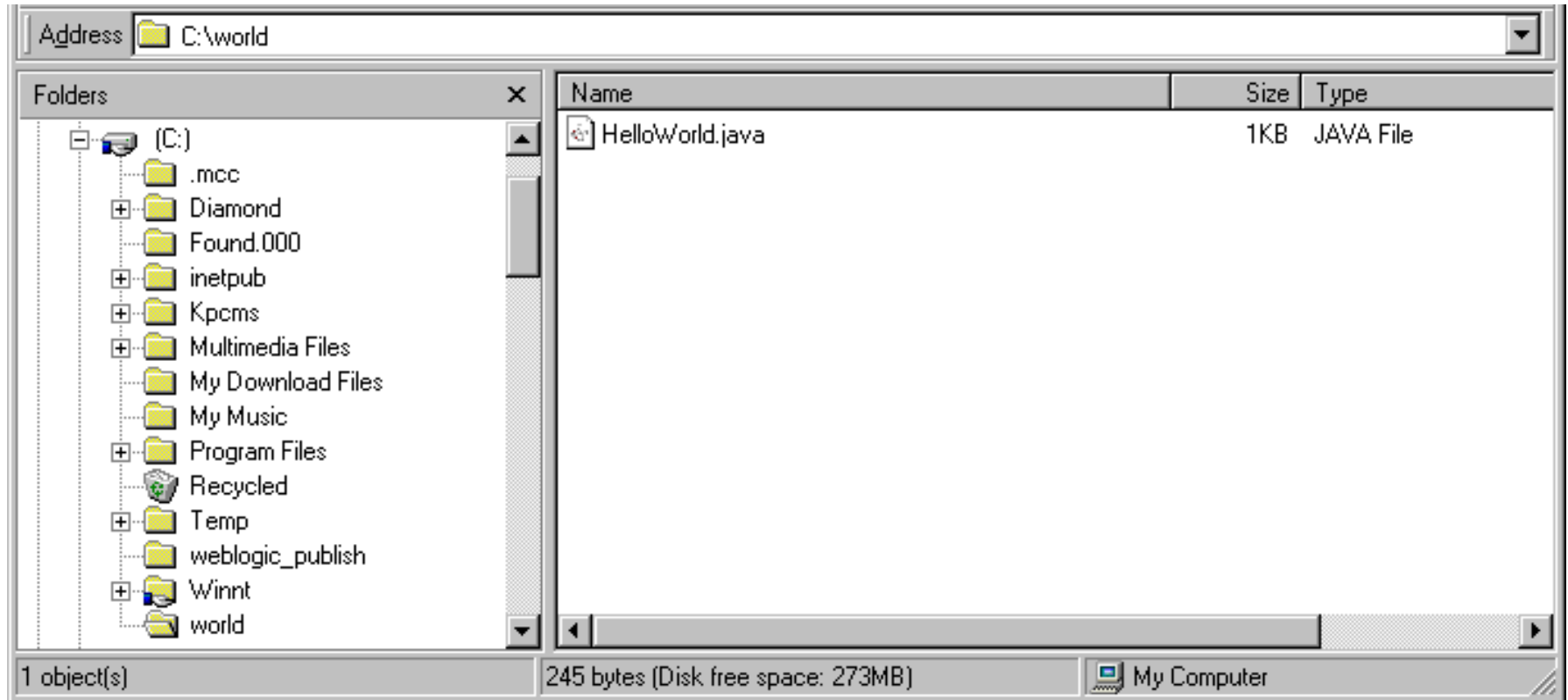
}

}

Creation of New Package

One thing you must do after creating a package for the class is to create nested subdirectories to represent package hierarchy of the class. In our case, we have the **world** package, which requires only one directory. So, we create a directory **world** and put our HelloWorld.java into it.

Creation of New Package



Setting up the CLASSPATH

From the above figure, we put the package world under C:. So we just set our CLASSPATH as:

set CLASSPATH=.;C:\;

Note: If you used to play around with DOS or UNIX, you may be familiar with . (dot) and .. (dot dot). We use . as an alias for the current directory and .. for the parent directory. In our CLASSPATH we include this . for convenient reason. Java will find our class file not only from C: directory but from the current directory as well. Also, we use ; (semicolon) to separate the directory location in case we keep class files in many places.

Setting up the CLASSPATH

When compiling HelloWorld class, we just go to the world directory and type the command:

```
C:\world\javac HelloWorld.java
```

If you try to run this HelloWorld using **java HelloWorld**, you will get the following error:

```
C:\world>java HelloWorld
```

```
Exception           in           thread           "main"
```

```
java.lang.NoClassDefFoundError: HelloWorld
```

```
(wrong name: world/HelloWorld)
```

Setting up the CLASSPATH

The reason is right now the HelloWorld class belongs to the package world. If we want to run it, we have to tell JVM about its **fully-qualified class name** (world.HelloWorld) instead of its plain class name (HelloWorld).

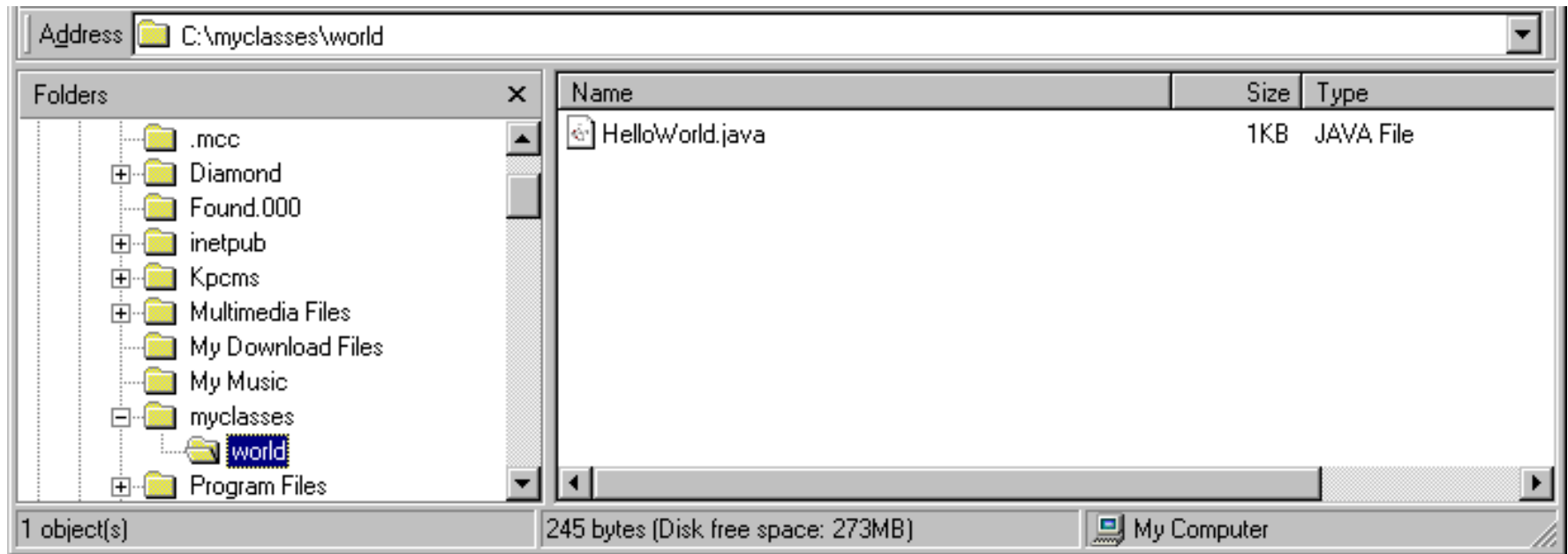
```
C:\world>java world.HelloWorld
```

```
C:\world>Hello World
```

Setting up the CLASSPATH

Now to make this example more understandable, let's put the HelloWorld class along with its package (world) be under **C:\myclasses** directory instead. The new location of our HelloWorld should be as shown in Figure.

Setting up the CLASSPATH



Setting up the CLASSPATH

We just changed the location of the package from

C:\world\HelloWorld.java

to

C:\myclasses\world\HelloWorld.java.

Our CLASSPATH then needs to be changed to point to the new location of the package world accordingly.

Setting up the CLASSPATH

```
set CLASSPATH=.;C:\myclasses;
```

Thus, Java will look for java classes from the current directory and **C:\myclasses** directory instead. Someone may ask "Do we have to run the HelloWorld at the directory that we store its class file everytime?". The answer is NO. We can run the HelloWorld from anywhere as long as we still include the package world in the CLASSPATH. For example,

Setting up the CLASSPATH

```
C:\>set CLASSPATH=.;C:\;
```

```
C:\>set CLASSPATH // see what we have
```

```
CLASSPATH=.;C:\;
```

```
C:\>cd world
```

```
C:\world>java world.HelloWorld
```

```
Hello World
```

```
C:\world>cd ..
```

```
C:\>java world.HelloWorld
```

```
Hello World
```

Subpackage (package inside another package)

Assume we have another file called HelloMoon.java. We want to store it in a subpackage "moon", which stays inside package world. The HelloMoon class should look something like this:

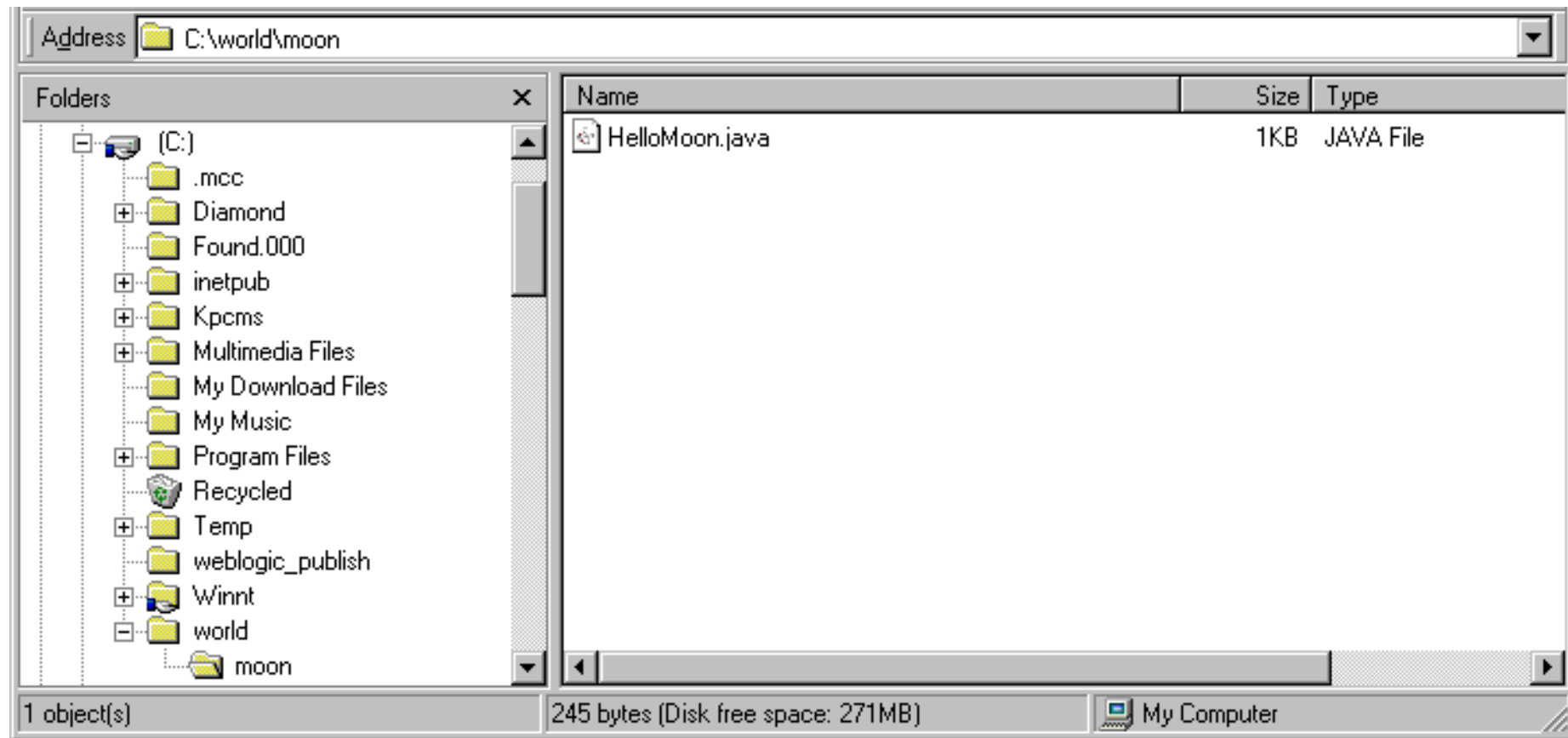
Subpackage (package inside another package)

```
package world.moon;  
public class HelloMoon  
{  
    private String holeName = "rabbit hole";  
    public getHoleName()  
    {  
        return hole;  
    }  
    public setHole(String holeName)  
    {  
        this.holeName = holeName;  
    }  
}
```

Subpackage (package inside another package)

If we store the package world under C: as before, the HelloMoon.java would be c:\world\moon\HelloMoon.java as shown in Figure below :

Subpackage (package inside another package)



Subpackage (package inside another package)

If we store the package world under C: as before, the HelloMoon.java would be c:\world\moon\HelloMoon.java as shown in Figure below :

Subpackage (package inside another package)

Although we add a subpackage under package world, we still don't have to change anything in our CLASSPATH. However, when we want to reference to the HelloMoon class, we have to use world.moon.HelloMoon as its fully-qualified class name.

How to use Package

There are 2 ways in order to use the public classes stored in package.

1. Declare the fully-qualified class name.

For example,

```
world.HelloWorld helloWorld = new world.HelloWorld();  
world.moon.HelloMoon helloMoon = new world.moon.HelloMoon();  
String holeName = helloMoon.getHoleName();
```

How to use Package

2) Use an "import" keyword:

```
import world.*;
```

// we can call any public classes inside the world package

```
import world.moon.*;
```

*// we can call any public classes inside the **world.moon** package*

```
import java.util.*;
```

*// import all public classes from **java.util** package*

```
import java.util.Hashtable;
```

*// import only **Hashtable** class (not all classes in java.util package)*

How to use Package

Thus, the code that we use to call the HelloWorld and HelloMoon class should be ...

```
HelloWorld helloWorld = new HelloWorld();
```

```
    // don't have to explicitly specify  
world.HelloWorld anymore
```

```
HelloMoon helloMoon = new HelloMoon();
```

```
    // don't have to explicitly specify  
world.moon.HelloMoon anymore ...
```

[ex\ex22.java](#)

Access Modifiers

Class Modifiers

Keyword	Meaning
abstract	Must be extended
final	Cannot be extended
public	can be accessed by any other class but if the keyword is missing then access is limited to the current package

Access Modifiers

Method Modifiers

Keyword	Meaning
abstract	Must be overridden
final	Must not be overridden
native	Implemented in machine code used by the host cpu not by the Java ByteCode
private	Can be invoked only by the same class
protected	Can be invoked only by code of subclass of same package
public	Can be invoked by any other class
static	Not an instance variable
synchronized	acquires a lock when begins execution

Access Modifiers

Variable Modifiers

Keyword	Meaning
final	It is a constant
private	Can be accessed only by the same class only
protected	Can be accessed only by the class in which it is declared and sub classes of the same package
public	Can be accessed by any other class
static	is not an instance variable

Access Modifiers

Access Modifier and Access Location	Public	Protected	Friendly (default)	Private protected	private
Same class	Yes	Yes	Yes	Yes	Yes
Subclass in same package	Yes	Yes	Yes	Yes	No
Other classes of same package	Yes	Yes	Yes	No	No
Subclass in other package	Yes	Yes	No	Yes	No
Non-subclass of other packages	Yes	No	No	No	No

Access Modifiers

- **Points to be remember (Summary)**
 - Use **public** if the field is visible everywhere
 - Use **protected** if the field is to be visible everywhere in current package and subclasses of other packages.
 - Use **default** if the field is to be visible everywhere in the **current package only**.
 - Use **private protected** if the field if the field is to be visible only in subclasses **regardless of package**.
 - Use **private** if the field is **not to be visible anywhere except its own class**.

[ex\java-access-modifiers.zip](#)



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
Use of java.lang package



Introduction

In java API one of the basic package is java.lang package. This package contains the core classes and interfaces used in the java language. These includes classes and interfaces like System, String, StringBuffer, StringBuilder, CharSequence, Comparable, Math and Wrapper classes.

Introduction

From our earlier discussions we know that the relational operators like $<$, $>$, $\leq$, $\geq$ work only for numeric data types. i.e. the comparison is not available for reference data types. On the other hand we don't require to compare all things in terms of less than and greater than. E.g. for boolean data type it is quite logical that we cannot have comparison between two boolean values for greater than or less than.

Introduction

It is now up to us (i.e. on programmers) that when we create a new data type, by defining a class, he should be able to decide whether the instances of the class which is being defined can be compared for greater than or less than relations. This is possible with the help of Comparable interface.

Comparable interface

Comparable interface is always used to indicate that instances of a particular class have a natural ordering. The logic for making the comparison depends on the class. Each class which is being defined as Comparable has its own logic for comparison. This can be specified by implementing the `compareTo` method from the Comparable interface.

Comparable interface

The Comparable interface can be defined with compareTo() method as follows :

```
public interface Comparable
{
    public int compareTo(Object o)
    {
        .....
    }
}
```

Comparable interface

The `compareTo()` method returns an `int` value. This value is interpreted to indicate whether the current object being compared to another instance (passed as parameter) is greater than, less than or equal to the other object. E.g. if we have two instances `x` and `y` of a given class which implements the `Comparable` interface, then the `int` value returned from the invocation of `x.compareTo(y)` will be interpreted as :

Comparable interface

- (1) If the returned value is negative then x is considered to precede y i.e. x is less than y.
- (2) If the returned value is positive then x is considered to succeed y i.e x is greater than y.
- (3) If the returned value is zero then x and y are considered to be equivalent i.e. x is equal to y.

[ex\ex39.java](#)

Comparator interface

The sort method for sorting the elements in an Object array is overloaded in the Arrays class. This method takes two parameters, one is the Object array and the other is of type Comparator. Comparator is an interface of java.util package. In case of Comparable interface we have the compareTo() method which takes only one parameter. The Comparator interface has only one method called compare() which takes two parameters.

Comparator interface

```
public interface Comparator  
{  
    public int compare(Object o1, Object o2)  
}
```

The compare method returns an int value. This value is interpreted to indicate whether the first object being compared to the second object is greater than, less than or equal to the second object. e. g. we have two instances x and y then the return int value from the invocation of compare(x,y) will be interpreted as follows :

Comparator interface

- (1) If the value is positive then x is considered to precede y.
- (2) If the value is negative then x is considered to succeed y.
- (3) If the value is zero then x and y are considered as equivalent.

Remember that after all you have to use the `compareTo()` method inside the `compare` method to compare individual objects.

StringBuilder Class

As we know that strings are immutable, you may want to use the StringBuilder class if you're going to alter the String in the code.

The StringBuilder class can be seen as a mutable String object which allocates more memory when its content is altered.

A StringBuilder object has an attribute called capacity that determines how many bytes that are allocated in the buffer.

[ex\ex40.java](#)

[ex\ex41.java](#)

[ex\ex42.java](#)



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
Strings



Introduction

String is a very crucial element in any of the programming language. We know that string is a group of characters.

Here in java an object of the String class represents a string of characters. The String class belongs to the java.lang package, which is built into Java. Like other classes, String has constructors and methods. Unlike other classes, String has two operators, + and += (used for concatenation).

Introduction

The string text may include “escape” characters like backslash codes

For example:

\\ stands for \

\n stands for the newline character

e.g.

```
String s1 = "Biology";
```

```
String s2 = "C:\\jdk1.4\\docs";
```

```
String s3 = "Hello\n";
```

Introduction

Remember that Strings in java is immutable object means that once created, a string cannot be changed: none of its methods changes the string. Such objects are called *immutable*.

Immutable objects are convenient because several references can point to the same object safely: there is no danger of changing an object through one reference without the others being aware of the change.

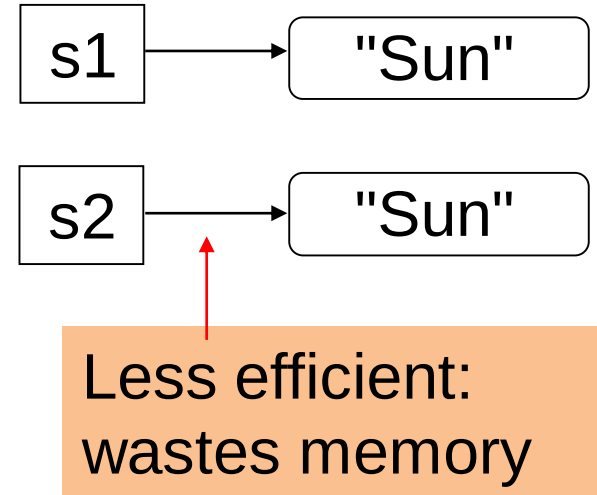
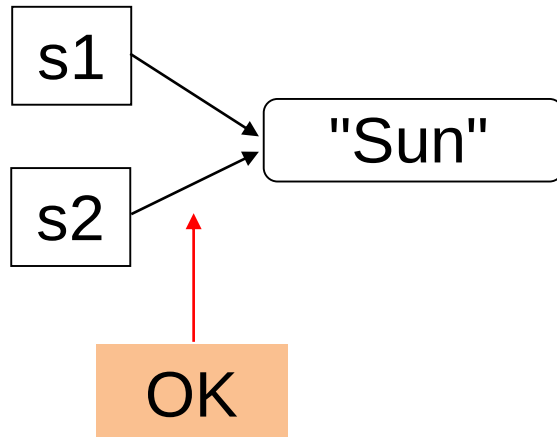
Introduction

Remember that Strings in java is immutable object means that once created, a string cannot be changed: none of its methods changes the string. Such objects are called *immutable*.

Immutable objects are convenient because several references can point to the same object safely: there is no danger of changing an object through one reference without the others being aware of the change.

Advantage of using Strings

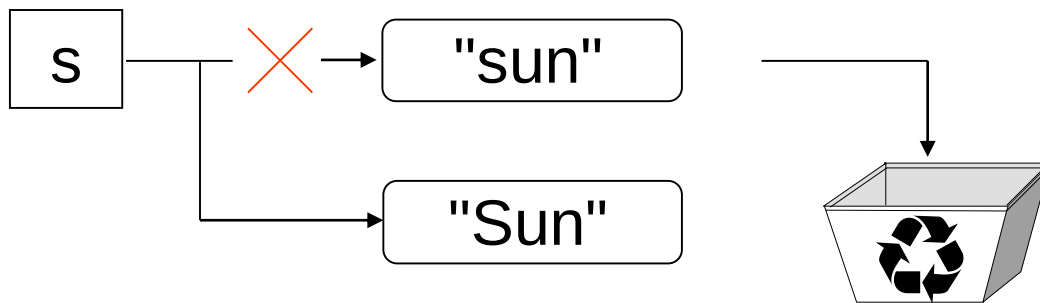
more efficient, no need to copy.



Disadvantage of using Strings

Less efficient — you need to create a new string and throw away the old one for every small change.

```
String s = "sun";  
char ch = Character.toUpperCase(s.charAt (0));  
s = ch + s.substring (1);
```



Empty String

An empty string has no characters; its length is 0.

```
String s1 = "";  
String s2 = new String();
```

Empty strings



Remember that : uninitialized string is different from empty string

```
private String errorMsg;
```

errorMsg
is null



Various methods of String Class

Method	Usage	Example
length()	Returns the no. of characters in the String	String str="Computer" str.length(); Will return 8.
charAt(n)	Returns the nth character from given string. (Remember that counts from zero)	String str="Computer" str.charAt(3); Will return p.
substring()	We can use this method in two ways. (1) by passing two values starting and ending (2) by passing only starting value so that it goes upto end.	String str="Computer" str.substring(3,5); Will return pu. String str="Computer" str.substring(3); Will return puter
concat()	Concates two strings and stores the result in new string	String s1 = "abc" String s2 = "xyz" String s3=s1.concat(s2) Result of s3 is "abcxyz"

Various methods of String Class

Method	Usage	Example
indexOf()	Returns the position of the character from beginning or from the given position	

0 8 11 15

String date = "July 5, 2012 1:28:19 PM";

Returns:

date.indexOf('J'); 0

date.indexOf('2'); 8

date.indexOf("2012"); 8

date.indexOf('2', 9); 11

(starts searching
at position 9)

date.indexOf("2020"); -1

date.lastIndexOf('2'); 15

(not found)

Various methods of String Class

Method	Usage	Example
<code>equals()</code>	Returns true if both the strings are equal.	<code>boolean b = s1.equals(s2);</code>
<code>equalsIgnoreCase()</code>	Returns true if both strings are equal (without case sensitivity)	
<code>compareTo</code>	Returns the difference between two strings.	
<code>compareToIgnoreCase()</code>	Returns the difference between two strings (ignoring case)	
<code>trim()</code>	Removes white spaces from both the sides of the string and return the new string.	
<code>replace()</code>	Replace the given character from the given string with another character.	<code>String str="Replace Region"; System.out.println(str.replace('R','A')); Output : Aeplace Aegion.</code>

Various methods of String Class

Method	Usage	Example
toUpperCase()	Converts given string into uppercase letters	
toLowerCase()	Converts given string into lowercase letters	

Remember that we have to write like `s1 = s1.toUpperCase()` then and then the string `s1` will be converted to uppercase but if we just write `s1.toUpperCase()` then it has no effect.

Numbers to Strings

Three ways to convert a number into a string:

1.

```
String s = "" + num;
```

2.

```
String s = Integer.toString (i);
```

```
String s = Double.toString (d);
```

3.

```
String s = String.valueOf (num);
```

[ex\ex14.java](#)

Integer and **Double** are “wrapper” classes from **java.lang** that represent numbers as objects. They also provide useful static methods.

Numbers to Strings

The DecimalFormat class can be used for formatting numbers into strings.

```
import java.text.DecimalFormat;  
...  
    DecimalFormat money =  
        new DecimalFormat("0.00");  
...  
    double amt = 56.7381;  
...  
    String s = money.format (amt);
```

56.7381



"56.74"

Numbers to Strings

Java 5.0 added `printf` and `format` methods:

```
int m = 5, d = 19, y = 2007;  
double amt = 123.5;
```

```
System.out.printf (  
    "Date: %02d/%02d/%d Amount = %7.2f\n", m, d, y, amt);
```

```
String s = String.format(  
    "Date: %02d/%02d/%d Amount = %7.2f\n", m, d, y, amt);
```

Displays,
sets **s** to:

"Date: 05/19/2007 Amount 123.50"

The StringBuffer class

Represents a string of characters as a mutable object

Constructors:

`StringBuffer()` // empty StringBuffer of the default capacity

`StringBuffer(n)` // empty StringBuffer of a given capacity

`StringBuffer(str)` // converts str into a StringBuffer

Adds `setCharAt`, `insert`, `append`, and `delete` methods

The `toString` method converts this StringBuffer into a String

[ex\ex15.java](#)

The **StringBuilder** class

Strings are immutable (can't be changed), so manipulating strings often causes many temporary string objects to be created, which can be quite inefficient.

StringBuilder (or **StringBuffer**) are better choices in these cases. Their `toString()` method can be used to get a `String` value. It isn't necessary to create them with a specific size, it will expand as necessary.

The StringBuilder class

StringBuilder is basically identical to the older StringBuffer, but is slightly faster because it isn't synchronized.

Constructors

```
sb = new StringBuilder()
```

Creates empty StringBuilder

```
sb = new StringBuffer(n)
```

Creates empty StringBuilder with initial capacity n.

```
sb = new StringBuffer(s)
```

Creates StringBuilder with value initialized to String s.



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
Vectors and Wrappers



Introduction to Vector Class

Since beginning we know that java is pure Object Oriented Programming Language. So everything in java is Objects. We also know that java supports variable arguments feature.

This feature also achieved in java through Vector class of java.util package.

This class is useful to create generic dynamic array known as vector which can hold objects of any type and any number.

Note that : the objects do not need to be homogeneous.

Introduction to Vector Class

We can declare Vector like :

Vector v = new Vector(); // Declaration
without size

Vector v1 = new Vector(3); // Declaration with
size of 3

Remember that : when we declare vector without size then it can occupy any no. of items. Even when size is specified this can be overlooked and we can store any no. of items.

Benefits of Vector over Array

It is convenient to use vectors to store Objects

A vector can be used to store a list of objects that may vary in size.

We can add and delete objects as and when we require.

Note that : we are not allow to store simple variables in vector so we need to convert the simple variables to objects by using Wrapper Class.

Methods of Vector Class

Method	Description
<code>list.addElement(item)</code>	Add the item specified to the list at the end.
<code>boolean contains(Object e)</code>	Returns true if the e contained in the list otherwise returns false
<code>list.elementAt(10)</code>	Gives the name of the 10 th object
<code>boolean isEmpty()</code>	Returns true if the list is empty otherwise returns false.
<code>list.size()</code>	Gives the number of object present in the list
<code>list.removeElement(item)</code>	Removes the specific element from the list.
<code>list.removeElementAt(n)</code>	Removes the item stored in nth position of the list
<code>list.removeAllElements()</code>	Removes all the elements from the list.
<code>list.copyInto(array)</code>	Copies all the items from list to array
<code>list.insertElementAt(item,n)</code>	Insert the item at nth position

[ex\ex16.java](#)

Introduction to Wrapper Class

Since beginning we know that java is pure Object Oriented Programming Language. So everything in java is Objects. When we need to convert simple data types into objects and objects to simple data types then we need Wrapper Class.

Introduction to Wrapper Class

Primitive Data Type	Wrapper Class
boolean	Boolean
byte	Byte
char	Character
short	Short
int	Integer
long	Long
float	Float
double	Double

Various Methods of Wrapper Class

Method	Description
Conversion from Primitive Numbers to Object Numbers	
Integer I = new Integer(i)	Primitive integer to Integer Object
Float F = new Float(f)	Primitive float to Float Object
Double D = new Double(d)	Primitive double to Double Object
Long L = new Long(l)	Primitive long to Long Object
Conversion from Object Numbers to Primitive Numbers	
int i = I.intValue()	Object to primitive integer
float f = F.floatValue()	Object to primitive float
long l = L.longValue()	Object to primitive long
double d = D.doubleValue()	Object to primitive double

Various Methods of Wrapper Class

Method	Description
Conversion from Number to String	
<code>str=Integer.toString(i)</code>	Primitive integer to string
<code>str=Float.toString(f)</code>	Primitive float to string
<code>str=Double.toString(d)</code>	Primitive double to string
<code>str=Long.toString(l)</code>	Primitive long to string
Conversion from String to Number Objects	
<code>D=Double.valueOf(str)</code>	Converts string to Double Object
<code>F=Float.valueOf(str)</code>	Converts string to Float Object
<code>I=Integer.valueOf(str)</code>	Converts string to Integer Object
<code>L=Long.valueOf(str)</code>	Converts string to Long Object

Various Methods of Wrapper Class

Method	Description
Conversion from String to Primitive numbers using Parsing	
<code>int i=Integer.parseInt(str)</code>	Converts string to primitive integer
<code>long l = Long.parseLong(str)</code>	Converts string to primitive long
<code>float f = Float.parseFloat(str)</code>	Converts string to primitive float
<code>double d = Double.parseDouble(str)</code>	Converts string to primitive double

[ex\ex17.java](#)



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 3
Exceptions



Introduction

During the execution of an application, there are many times when an error condition arises within the application. In java, all kinds of error conditions are called as exceptions. In other words we can say that **In General, it is in the rare case that our program will run successfully in its first attempt.**

There are mainly two types of exceptions :

(1) Compile Time Exceptions (Syntax Errors)

(2) Run Time Exceptions (Logical Errors)

Introduction

(1) Compile Time Errors :

All the syntax Errors are known as Compile Time Errors. These types of errors are detected and displayed by the java compiler therefore they are known as Compile Time Errors. This type of errors can be solved before running the program so that after solving this type of errors we are sure that our program is syntactically correct.

Remember that : Whenever the compiler will display the error it will not generate the .class file.

Introduction

(1) Compile Time Errors Examples :

- Missing Semicolons
- Missing Brackets
- Misspell of Keywords of Variables
- Missing of double quotes in strings
- Use of undeclared variables
- Incompatible type conversion / assignment
- use of = inplace of ==
- use of non initialized variable in keyboard reading

Introduction

(2) Run Time Errors :

Run time errors are those types of errors in which the program may compiled successfully and the .class file also generated but when we run that program it will produce the wrong result.

This will happened due to poor understanding of the logic or sometimes due to misuse of the system resources.

Introduction

(2) Run Time Errors Examples :

- Dividing an integer by Zero
- Accessing the element out of bounds of Array
- Trying to store the value of incompatible type into array
- Passing the parameter which is not compatible for the method.
- Trying to illegally change the state of Thread.
- Use of Negative size of Array.
- Conversion of invalid string to number

[ex\ex43.java](#)

Exception Handling

Generally an Exception is an error caused by run-time error in the program. If this types of errors (run time errors) are not handled properly the interpreter will generate the error as shown in previous example. To handle this situation java has provided us a special mechanism called Exception Handling. The purpose of this mechanism is to handle the exceptional circumstances.

Exception Handling

The mechanism for Exception handling can be summarize in the following steps:

- (1) Find the problem (Hit)
- (2) Inform that an error has occurred (Throw)
- (3) Receive the error information (Catch)
- (4) Take the corrective Actions (Handle)

For this purpose java has given us various built in Exceptions Classes to handle some common exceptions.

Some Common Java Exceptions

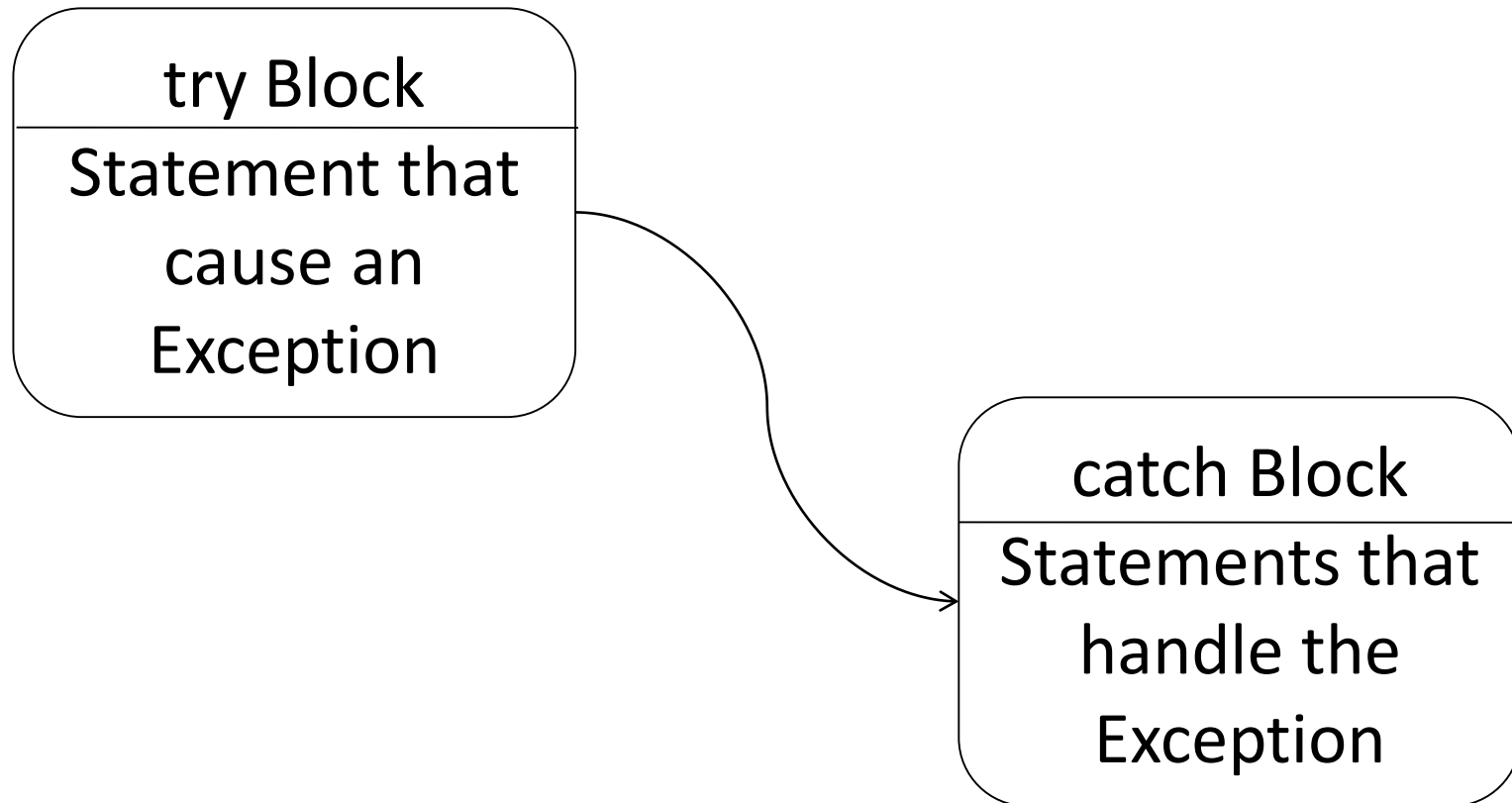
Exception	Cause of Exception
ArithmeticException	Caused by mathematical Errors such as division by Zero
ArrayIndexOutOfBoundsException	Caused by bad array indexes
ArrayStoreException	Caused when program try to store wrong type of data in array.
FileNotFoundException	Caused by an attempt to access a nonexistent file.
IOException	Caused by general I / O failures.
NullPointerException	Caused by referencing to a null object
NumberFormatException	Caused when conversion between string and number fails

Some Common Java Exceptions

Exception	Cause of Exception
OutOfMemoryException	Caused when there is no enough memory to allocate a new object
SecurityException	Caused when an applet tries to perform an action not allowed by the browser's security settings.
StackOverflowException	Caused when the system runs out of stack space
StringIndexOutOfBoundsException	Caused when a program attempts to access a non existent character position in a string.

Exception Handling Mechanism

Following figure can illustrate the Exception Handling Mechanism



Exception Handling Mechanism

Java uses a keyword “try” to preface a block of code that is likely to cause an error condition and which is liable to “throw” the exception.

A catch block will be defined by the keyword “catch” which contains the statements which handles the situation thrown by the try block.

Remember that : when you use try u have to use at least one catch otherwise it will generate the error like try without catch.

[ex\ex44.java](#)

[ex\ex45.java](#)

Use of Multiple Catch Statements

In java we can use multiple catch statements if we think that particular statement block will generate multiple type of errors.

e.g. if we think that a group of statements i.e. a block will generate either divide by zero or array index out of bounds errors then we can use multiple catch statements.

[ex\ex46.java](#)

Use of finally

Java supports another statement named “finally” which is useful to handle an exception that is not caught by any previous catch statements.

finally statement is used to handle any exception generated within the try block.

Remember that when we use finally it is guaranteed that it will be executed, regardless of whether or not exception is thrown.

[ex\ex50.java](#)

Throwing our own Exception

Sometimes we need to throw our own exception we can do this by using the keyword “throw”.

Syntax :

```
throw new <throwable subclass>
```

Example :

```
throw new ArithmeticException();
```

We can also create our own class for Exception which must extends the Exception class.

[ex\ex47.java](#)

Difference between throw and throws keywords

Whenever we want to force an exception then we use throw keyword. the **throw** keyword (**note the singular form**) is used to force an exception. It can also pass a custom message to your exception handling module. Moreover **throw** keyword can also be used to pass a custom message to the exception handling module i.e. the message which we want to be printed.

For instance in the next example we have used -
throw new MyException ("can't be divided by zero");

Difference between throw and throws keywords

Whereas when we know that a particular exception may be thrown or to pass a possible exception then we use **throws** keyword. **Point to note here is that the Java compiler very well knows about the exceptions thrown by some methods so it insists us to handle them.** We can also use **throws clause** on the surrounding method instead of **try and catch exception handler**. For instance in the below example we have used the following clause which will pass the error up to the next level.

Difference between throw and throws keywords

```
static int divide (int first, int second)  
                                throws MyException  
{  
    .....  
}
```

[ex\ex48.java](#)

Checked and Unchecked Classes

At compile time, the java compiler checks that a program contains handlers for checked exceptions. Java compiler analyzes by which checked exceptions can result from execution of a method or constructor. For each checked exception which is a possible result, the throws clause for the method or constructor must mention the class or its super classes of that exception.

Checked and Unchecked Classes

The class `RuntimeException` and its subclasses, and the class `Error` and its subclasses are unchecked exceptions classes. Because the compiler doesn't force them to be declared in the `throws` clause. All the other exception classes that are part of `Throwable` hierarchy are checked exceptions.

Checked and Unchecked Classes

Now let us see a small discussion on why exceptions are classified as checked exceptions and unchecked exceptions.

Those unchecked exception classes which are the error classes (Error and its subclasses) are exempted from compile-time checking in java because they can occur at many points in the program and recovery from them is difficult or impossible.

Checked and Unchecked Classes

Example: OutOfMemoryError

Thrown when the Java Virtual Machine cannot allocate an object because it is out of memory, and no more memory could be made available by the garbage collector.

The runtime exception classes (RuntimeException and its subclasses) are exempted from compile-time checking because, in the judgment of the designers of the Java, having to declare such exceptions would not aid significantly in establishing the correctness of programs.

Checked and Unchecked Classes

Example: NullPointerException

Thrown when an application attempts to use null in a case where an object is required.

So the java compiler doesn't forces them to be declared in the above two cases.

Use of Throws with Overriding methods

Remember that : An override method cannot introduce new kinds of throws for checked exceptions. Just like the return type of an overriding method cannot be changed to return anything which is not according to the contract of the method being overridden, similarly the throws list of overriding methods cannot specify any new kind of checked exception which are not in the contract of the method being overridden.

The Throwable Class

The throwable class is the base class for all kind of exceptions. An instance of throwable class manages (a) a ***message***, which is taken as a parameter in the constructor, (b) a ***cause***, which can be taken as a parameter in the constructor or may be set only once using the `initCause()` method and (c) a ***stack trace***, which is initially set from the constructor using the `fillInStackTrace()` method.

The Throwable Class

Various methods of Throwable class are as follows :

- (1) public String getMessage()
- (2) public void initCause(Throwable cause)
- (3) public Throwable getCause()
- (4) public void fillInStackTrace()
- (5) public StackTraceElement[] getStackTrace()
- (6) public void printStackTrace()
- (7) public String toString()

[ex\ex48.java](#)

Exception Chaining

Whenever in a program the first exception causes another exception, that is termed as Chained Exception. Java provides new functionality for chaining exceptions. Exception chaining (also known as "nesting exception") is a technique for handling the exception, which occur one after another i.e. most of the time is given by an application to response to an exception by throwing another exception. Typically the second exception is caused by the first exception. Therefore chained exceptions help the programmer to know when one exception causes another.

Exception Chaining

The syntax for using a chained exception is as follows in which a new `TestException` exception is created with the attached cause when an `IOException` is caught. Thus the chain exception is thrown to next level of exception handler.

```
try
{
}
catch (IOException e)
{
throw new TestException("Other IOException", e);
}
```

[ex\ex49.java](#)



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 3
Member Types



Introduction

What are the members of a class ? But obvious the answer is Variables, Methods, Constructors, Initializer blocks etc.

Over and above these types of members in the class which we have used so far, it is also possible to have a class or an interface as a member of class. The classes defined inside another class are known as nested classes.

Introduction

What is a class ? Class is a data type. So when we define a nested class it is just another data type. Let us say we have to define a class A inside some package pkg and define the class B as a member of the class A; then the member class is another class whose name would become pkg.A.B. These member classes are like any other class.

Introduction

e.g.

```
package pkg;  
public class A  
{  
    class B  
    {  
        ....  
    }  
}
```

Introduction

Remember that : if in the above example we declare the class B as static then it is known as a top-level nested class.

Top-level Nested Class

The static member class is known as top-level nested class. These classes are similar to any other class except that the classname is now **<enclosing-class-name>.<member-class-name>** and their accessibility can be declared to be private or protected over and above the public and default, which are available for other classes that are members of a package.

Top-level Nested Class

There is one more advantage of creating a static member class instead of a class outside the class. The member class and the enclosing class share a trust relationship in that they both can access each other's members, including private members also.

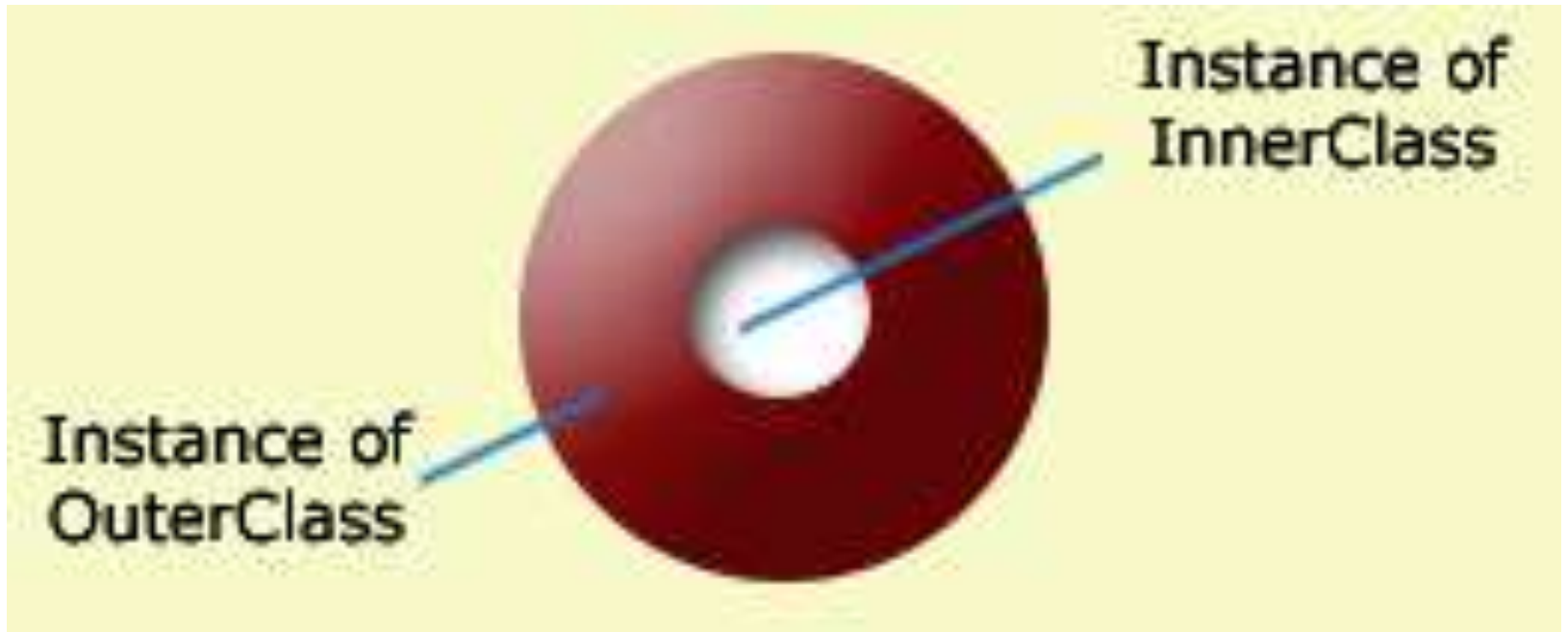
Remember that : We can add all the members in this top-level nested class that we can have all kinds of members that are allowed for the package level class.

Inner Classes

In simple words Inner class is a class declared inside the other class. The main difference between inner class and the top-level nested class is, in top-level nested class we declare the class as static while here the class will not be declared as static but it is declared like a normal class.

Methods and variables of an inner class can be access directly by the object of outer class. The figure given below illustrates it logically.

Inner Classes



[ex\ex51.java](#)

Local Classes

Just like we can declare variables locally within a method, constructor or an initializer block, we can also define a class inside a method, constructor or an initializer block. These classes are known as local classes.

Remember that : These classes are data types known within the block where they are defined.

[ex\ex52.java](#)

Anonymous Inner Class

Till now, we have seen that we can have classes as members of a class, just like instance variables. We can also define the class locally, means within the method also.

Now we will transfer to another type of nested class called as **Anonymous Inner Class**.

In Anonymous Inner Class, it does not have a name (Name is not explicitly defined).

Anonymous Inner Class

Now, the question is if the class does not have a name then how do we create instance of such a class ?

The answer is anonymous inner class are defined and created at the same place. Here in other words we can say Anonymous class is a part of an expression.

When we need any instance of that class then we could do it by starting with the keyword `new` followed by the invocation of the constructor followed by a block in which we define the class.

Anonymous Inner Class

Generally in java Anonymous Inner Classes are used while doing applet programming. In applets we can use anonymous inner class while developing applications with mouse programming.

We will see the examples of anonymous inner class while learning applets.



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 3
Classes from java.util package



Introduction

java.util package has lots of classes and interfaces. These are mainly related to the **collection framework (Our next topic)**. Apart from collection framework it has many utility classes related to date and time.

Mainly the java.util package is known as utility package and it contains various classes which will become handy tool for us while developing the applications.

Remember that : some of classes are not in our syllabi even though I have taken them.

The Random Class

The Random class allows us to generate random double, float, int or long numbers. These type of numbers are very useful when we are dealing with the real-world simulation systems.

This class provides two constructors which are as follows :

(1) Random()

(2) Random(long x)

Here x is a value to initialize the random number generator while in first case current time will be used as x

The Random Class

Various methods defined by the class are as follows :

Method	Description
double nextDouble()	Returns a random double value
float nextFloat()	Returns a random float value
int nextInt()	Returns a random int value
Long getLong()	Returns a random long value

[ex\ex53.java](#)

- (1) Write a program to generate 10 random nos. and find sum and average of them**
- (2) Write a program to generate 10 random nos. and find its average and nos. more than avg.**

The Date Class

The Date class gives us the information about the date and time. It has following constructors

Date()

Date(long msec)

The first form returns an object that represents current date and time. The second form returns an object that represents date and time msec after the epoch. The epoch is defined as midnight on january 1, 1970 (GMT)

The Date Class

Various methods defined by the class are as follows :

Method	Description
boolean after(Date d)	Returns true if d is after the current date. Otherwise returns false.
boolean before(Date d)	Returns true if d is before the current date. Otherwise returns false.
boolean equals(Date d)	Returns true if d has the same value as the calling object date. Otherwise returns false.
long getTime()	Returns the number of milliseconds since the epoch
void setTime(long msec)	Sets the date and time of the current object to represent msec milliseconds since the epoch.
int compareTo(Date d)	Compares the two dates for ordering. The value 0 if the argument Date d is equal to this Date; a value less than 0 if this Date is before the Date d argument; and a value greater than 0 if this Date is after the Date argument.

The Date Class

[ex\ex54.java](#)

- (1) Write a program to Calculate no. of days since 1st January, 1970**
- (2) Write a program to Add 100 days to the current date and display the new date and time.**
- (3) Write a program to compare two date objects and give appropriate msg**
- (4) Write a program to set your and your friends birth date in two different objects and give appropriate msg regarding who is elder.**

The Calendar and GregorianCalendar Class

The Calendar class allows us to interpret date and time information. This class defines various int constants which are useful when we want to do stuff with calendars.

Remember that : Calendar class does not have public constructor instead of that we have to use getInstance() method to obtain the current date and time e.g.

Calendar getInstance()

The Calendar and Gregorian Calendar Class

Various constants defined by the class are as follows :

AM	AM_PM	APRIL	AUGUST	DATE
DAY_OF_MONTH	DAY_OF_WEEK	DAY_OF_WEEK_IN_MONTH	DAY_OF_YEAR	DECEMBER
ERA	FEBRUARY	FRIDAY	HOUR	HOUR_OF_DAY
JANUARY	JULY	JUNE	MARCH	MAY
MILLISECOND	MINUTE	MONDAY	MONTH	NOVEMBER
OCTOBER	PM	SATURDAY	SECOND	SEPTEMBER
SUNDAY	THURSDAY	TUESDAY	WEDNESDAY	WEEK_OF_MONTH
WEEK_OF_YEAR	YEAR			

The Calendar and GregorianCalendar Class

Various methods defined by the class are as follows :

Method	Description
void add(Constant, Value)	Performs the arithmetic operation on Calendar's constant. E.g. add(Calendar.DATE, -5). ex\ex55.java
boolean after(Object c)	Returns true if the invoking Calendar object contains a date later than the one specified by c. Otherwise returns false.
boolean before(Object c)	Returns true if the invoking Calendar object contains a date earlier than the one specified by c. Otherwise returns false.
boolean equals(Object c)	Returns true if the invoking Calendar object contains a date equal to the one specified by c. Otherwise returns false.
int get(int Constant)	Returns the value of one component of the invoking object. The component will be any of the constant given in the previous list. ex\ex56.java
Calendar getInstance()	Returns the default Calendar object

The Calendar and GregorianCalendar Class

Various methods defined by the class are as follows :

Method	Description
<code>int getActualMaximum(int const)</code>	Return the maximum value that this field could have, given the current date. For example, with the date "Feb 3, 1997" and the DAY_OF_MONTH field, the actual maximum would be 28; for "Feb 3, 1996" it s 29.
<code>int getActualMinimum(int const)</code>	Return the minimum value that this field could have, given the current date.
<code>void set(int const)</code>	Set various date and time constants for the invoking object ex\ex57.java

The GregorianCalendar Class

GregorianCalendar is a concrete subclass of Calendar and provides the standard calendar used by most of the world.

This class has various constructors which are as follows :

- (1) `GregorianCalendar()`
- (2) `GregorianCalendar(int y,int m,int d)`
- (3) `GregorianCalendar(int y,int m,int d,
int h,int m, int s)`
- (4) `GregorianCalendar(int y,int m, int d,
int h, int m)`

The Gregorian Calendar Class

The first form creates an object initialized with the current date and time. Other forms allows you to specify how various date and time components will initialized.

Remember that : Gregorian Calendar uses all the methods of Calendar class except that one new method introduce is isLeapYear() method which returns boolean value either true or false (check for leap year)

The Calendar and Gregorian Calendar Class

- (1) Write a program to display current year and check whether it is leap year or not ?**
- (2) Write a program to enter your birth date from keyboard and find whether your birth year was leap year or not ?**
- (3) Write a program to enter your birth date from keyboard and find no. of years, months and days from your birth date.**

The DateFormat Class

DateFormat is an abstract class for date/time formatting subclasses which formats date or time in a language-independent manner. In this class you can pass any formatting string as argument through which you can format your date and time.

[ex\ex58.java](#)

[ex\ex59.java](#)

The Stack Class

A Stack is like a bucket in which you can put elements one-by-one in sequence and retrieve elements from the bucket according to the sequence of the last entered element. Stack is a collection of data and follows the **LIFO (Last in, first out)** rule that mean you can insert the elements one-by-one in sequence and the last inserted element can be retrieved at once from the bucket. Elements are inserted and retrieved to/from the stack through the `push()` and `pop()` method.

The Stack Class

Various methods defined by the class are as follows :

Method	Description
boolean empty()	Returns true if the stack is empty. Otherwise returns false
Object peek()	Returns the object from the top of the stack but does not remove it from the stack
Object pop()	Returns the object from the top of the stack and also removes it from the stack
Object push(Object obj)	Pushes obj into the stack and also return the obj
Int Search(Object obj)	Search the stack for the obj. If not found it returns -1, otherwise returns the index at which obj is located. The top of stack is index 1.

[ex\ex60.java](#)

The Hashtable Class

Hash Table holds the records according to the unique key value. It stores the non-contiguous key for several values. Hash Table is created using an algorithm (hashing function) to store the key and value regarding to the key in the hash bucket. If you want to store the value for the new key and if that key is already exists in the hash bucket then the situation known as **collision** occurs which is the problem in the hash table i.e. maintained by the hashing function.

The Hashtable Class

The drawback is that hash tables require a little bit more memory, and that you can not use the normal list procedures for working with them.

In simple words we can say A hash table contains key and value. Both key and value are objects. The keys must be unique but the values need not be. Another name of this type of data structure is associative array.

Remember that : there is no size limit in hashtable.

The Hashtable Class

Various methods defined by the class are as follows :

Method	Description
boolean contains(Object v)	Returns true if the hash table contains v as one of its values. Otherwise returns false.
boolean containsKey (Object k)	Returns true if the hash table contains k as one of its keys otherwise returns false.
booleans containsValue(Object v)	Returns true if the hash table contains v as one of its values. Otherwise returns false.
Enumeration elements()	Returns an enumeration of the values
Object get(Object k)	Returns the value associated with the key k
Boolean isEmpty()	Returns true if the hashtable is empty otherwise returns false
Enumeration keys()	Returns an enumeration of the keys

The Hashtable Class

Various methods defined by the class are as follows :

Method	Description
Object put(Object k, Object v)	Puts a key / value pair into the hashtable. Here the key is k and the value is v.
Object remove(Object k)	Removes the key / value pair whose key is k
int size()	Returns the number of keys in the hashtable

[ex\ex61.java](#)

The StringTokenizer Class

Tokens can be used where we want to break an application into tokens. We have to break a String into tokens as well as we will know how many tokens has been generated. StringTokenizer is a class in java.util.package. We are using while loop to generate tokens.

The StringTokenizer Class

Various methods defined by the class are as follows :

Method	Description
int countTokens()	Returns the number of tokens in the string
boolean hasMoreTokens()	Returns true if there are more tokens. Otherwise returns false
String nextToken()	Returns the next token

[ex\ex62.java](#)



Thank You

Marwadi Education Foundation's Group of Institutions
Faculty of Computer Applications
MCA Sem- III

Fundamental of Java Programming
(630002)



Unit – 2
AutoBoxing and Unboxing



Introduction

This is a new feature introduced from JDK 1.5. It provides us the facility to handle primitive data types easily with the help of Wrapper class.

Through this feature we can easily convert our regular data types to Wrapper Class objects and Wrapper Class objects to Regular data types.

for e.g. we can write like

```
Double dx = 88.77;  
double dprim = dx;
```

[ex\ex18.java](#)

Introduction to Enumeration

Enumeration works just like array. Java provides the facility of enumerated types through enum keyword.

e.g. public class days

{

public static final int day_sunday=0;

public static final int day_monday=1;

}

will be same as

public enum day {sunday, monday};

[ex\ex19.java](#)



Thank You

--- Exception Hierarchy in Java

The exception hierarchy is the foundation of Java's robust mechanism for handling errors and exceptional conditions that arise during program execution.

Theory: The Throwable Root

All error and exception classes in Java are derived from the common superclass `java.lang.Throwable`. This class is the single root of the entire hierarchy.

A. Direct Subclass: `java.lang.Error`

Nature: Represents serious, unrecoverable problems that occur outside the control of the application (e.g., hardware malfunction, JVM internal errors).

Handling: They are unchecked; the compiler does not force you to handle them. When an Error occurs, the application typically terminates abruptly.

Examples: `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`.

B. Direct Subclass: `java.lang.Exception`

Nature: Represents conditions that a well-written application should anticipate and handle gracefully.

Sub-Categories:

Checked Exceptions:

Definition: Direct descendants of `Exception` (but not descendants of `RuntimeException`).

Handling: The compiler enforces handling. The method must either surround the problematic code with a try-catch block or declare the exception using the `throws` keyword in the method signature.

Examples: `IOException`, `SQLException`, `FileNotFoundException`.

Unchecked (Runtime) Exceptions:

Definition: Descendants of `java.lang.RuntimeException` (which itself is a subclass of `Exception`).

Handling: The compiler does not enforce handling. They are typically caused by programming logic errors (bugs) that could have been avoided (e.g., trying to access memory that doesn't exist, invalid data operations).

Examples: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.

Example: Checked vs. Unchecked Exception Handling
Java

```
import java.io.File;
import java.io.FileReader;
import java.io.FileNotFoundException;

public class ExceptionDemo {
    public static void main(String[] args) {
        // 1. Unchecked Exception (Logic Error: Division by zero)
        try {
            int result = 10 / 0; // ArithmeticException - Unchecked
        } catch (ArithmeticException e) {
            System.out.println("Unchecked Error Handled: Cannot divide by
zero.");
        }

        // 2. Checked Exception (External Problem: File not found)
```

```

        // The compiler requires a try-catch for FileNotFoundException.
        File file = new File("config.txt");
        try {
            FileReader fr = new FileReader(file); // Throws
FileNotFoundException - Checked
            System.out.println("File opened successfully.");
        } catch (FileNotFoundException e) {
            System.out.println("Checked Exception Handled: File not found. Path:
" + e.getMessage());
        }
    }
}

```

--- 2. User-Defined Packages

A package is a powerful mechanism for organizing related classes and interfaces, providing both structure and access control to your code.

Theory: Organization and Access

A. Core Purposes

Code Organization: Packages group related types, making large projects easier to manage, navigate, and maintain.

Name Resolution/Conflict Avoidance: They create a unique namespace for classes. This allows multiple programmers or libraries to define classes with the same name (e.g., Employee) as long as they reside in different packages (com.hr.Employee vs. com.finance.Employee).

Access Control: The default access modifier (often called package-private) restricts access to class members only to other classes within the same package. The protected modifier also uses package boundaries for limited inheritance access.

B. Package Creation and Naming

Declaration: The package statement must be the very first line of a Java source file (excluding comments and whitespace).

Convention: Package names are written in all lowercase letters to distinguish them from class names. Standard practice uses the reversed internet domain name for globally unique top-level names (e.g., com.companyname.project.module).

File System Mapping: The Java compiler requires that the package structure exactly matches the directory structure on the file system. A package named com.mycompany.app must reside in a directory structure like com/mycompany/app/.

C. Using Packages

The import Statement: Used to bring a class or all classes from another package into the current file's scope, allowing you to refer to the imported class by its simple name.

Single Class: import java.util.Scanner;

All Classes (Wildcard): import java.util.*; (Note: This does not import sub-packages).

Example: Creating and Importing a Package

1. Creating the Package Class (Calculator.java)
// mathlib/util/Calculator.java
package mathlib.util; // The package declaration

```

public class Calculator {
    public static int add(int a, int b) {

```

```

        return a + b;
    }
    public static int multiply(int a, int b) {
        return a * b;
    }
}

```

2. Using the Package Class (MainApp.java)

```

// MainApp.java
import mathlib.util.Calculator; // Import the class using the package name

public class MainApp {
    public static void main(String[] args) {
        // Use the imported class directly by its simple name
        int resultAdd = Calculator.add(10, 5);
        int resultMultiply = Calculator.multiply(10, 5);

        System.out.println("Addition Result: " + resultAdd);
        System.out.println("Multiplication Result: " + resultMultiply);
    }
}

```

---3. Varargs (Variable Arguments) in Java

Varargs is a feature introduced in Java 5 that allows a method to accept zero or more arguments of a specified type using a shorthand syntax.

Theory: Flexibility and Array Conversion

A. Syntax and Mechanism

Syntax: You define a varargs parameter using the ellipsis (...) after the data type: `dataType... variableName`.

Java

```

public static void printList(String... items) { ... }

```

Mechanism: When a varargs method is called, the variable arguments are automatically encapsulated by the Java compiler into a one-dimensional array of the specified type. Inside the method body, the parameter (items in the example above) is treated exactly like an array (`String[]`).

B. Key Rules

Zero or More: A varargs method can be called with no arguments, one argument, or multiple arguments.

Position Constraint: If a method has other parameters, the varargs parameter must be the last parameter in the method's argument list. This is necessary to allow the compiler to distinguish the fixed parameters from the variable ones.

Uniqueness: A method can only have one varargs parameter.

Compatibility: You can pass an actual array to a varargs method, as the mechanism is designed to handle this naturally.

Example: Calculating an Average with Varargs

Java

```

public class VarargsDemo {

    // Varargs must be the last parameter (after the fixed 'description'
    // parameter)
    public static double calculateAverage(String description, double... values)
    {
        System.out.println("\n--- " + description + " ---");
    }
}

```

```

    if (values.length == 0) {
        System.out.println("No values provided.");
        return 0.0;
    }

    double sum = 0.0;
    // 'values' is treated as a double array (double[])
    for (double val : values) {
        sum += val;
    }

    return sum / values.length;
}

public static void main(String[] args) {
    // 1. Passing multiple arguments
    double avg1 = calculateAverage("Scores for Test 1", 90.5, 85.0, 92.5);
    System.out.printf("Average 1: %.2f%n", avg1);

    // 2. Passing zero arguments
    double avg2 = calculateAverage("Scores for Test 2 (Empty)");
    System.out.printf("Average 2: %.2f%n", avg2);

    // 3. Passing a single argument
    double avg3 = calculateAverage("Score for Test 3", 77.0);
    System.out.printf("Average 3: %.2f%n", avg3);

    // 4. Passing an actual array
    double[] weeklyTemps = {25.5, 26.0, 24.5, 27.0};
    double avg4 = calculateAverage("Weekly Temperatures", weeklyTemps);
    System.out.printf("Average 4: %.2f%n", avg4);
}
}

```

--- Scanner Class

The Scanner Class breaks an input text character into tokens based on delimiters, and then converts them into primitive values or String. A Scanner instance is created by specifying some input source. This input source may be a fixed String or some source of bytes or characters. The Scanner class is a class in java.util, which allows the user to read values of various types. which allow us to read in numeric values from either the keyboard or file without having to convert them from strings and determine if there are more values to be read.

Through scanner class we can read data from the input device or from file. Actually there are so many constructors in the class but for us mainly two constructors are useful. (1) for Reading data from keyboard (2) for Reading data from file.

```

(1) Scanner in = new Scanner(System.in);
(2) Scanner inFile = new Scanner(new FileReader("myFile"));

```

Various methods of the class are as follows :

Method,	Usage
void close(),	Close the scanner.
boolean hasNext(),	Returns true if the scanner has next token to read otherwise returns false.
String next(),	Finds and returns the next complete token from this scanner.
boolean nextBoolean(),	Scans the next token of the input into a boolean value and returns that value.
byte nextByte(),	Scans the next token of the input as a byte.
double nextDouble(),	Scans the next token of the input as a double.
float nextFloat(),	Scans the next token of the input as a float.

int nextInt(), Scans the next token of the input as an int.
String nextLine(), Advances this scanner past the current line and returns
the input that was skipped.
long nextLong(), Scans the next token of the input as a long.
short nextShort(), Scans the next token of the input as a short.
Scanner reset(), Reset the Scanner.